COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Soluzione esercizi sui dizionari

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Esercizi sui dizionari

Indicazioni sulla consegna **3**
- Struttura del codice sorgente 4

Esercizi sui Dizionari **6**
- Primo Esercizio 6
- Secondo Esercizio 6
- Terzo Esercizio 6
- Quarto Esercizio 6
- Quinto Esercizio 6

Soluzione esercizi sui Dizionari **7**
- Primo Esercizio 7
- Secondo Esercizio 10
- Terzo Esercizio 12
- Quarto Esercizio 14
- Quinto Esercizio 16

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Attenzione: non utilizzare strutture complesse oltre alle tuple

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Esercizi sui Dizionari

Primo Esercizio

Progettare una funzione che accetti un numero indefinito di dizionari e restituisca un dizionario che è la concatenazione di tutti i dizionari indicati come parametro formale alla funzione. Scrivete uno script che utilizzi tale funzione.

```
Esempio:
     diz1 = {'v1':1,'v2':2,'v3':3}
     diz2 = {'v4':4,'v5':5,'v6':6}
     diz3 = {'v7':7,'v8':8}

     Dizionario restituito: {'v1': 1, 'v2': 2, 'v3': 3, 'v4': 4,
     'v5': 5, 'v6': 6, 'v7': 7, 'v8': 8}
```

Secondo Esercizio

Scrivete uno script Python per generare e stampare un dizionario che contenga un numero (compreso tra 1 e n) nella forma (x, x*x).

```
Esempio:
     n = 5
     Dizionario: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Terzo Esercizio

Scrivete un programma Python per ottenere il valore massimo e minimo in un dizionario.

Quarto Esercizio

Scrivete un programma Python per rimuovere i duplicati dal dizionario.

Quinto Esercizio

Scrivete un programma Python per creare un dizionario da una stringa. Le lettere della stringa rappresentano le chiavi, i valori rappresentano le occorrenze della chiave nella stringa

```
Esempio
     stringa = “ciao mamma”'
     Dizionario: {'c': 1, 'i': 1, 'a': 3, ‘o’: 1, ' ':1,'m': 3}
```

Soluzione esercizi sui Dizionari

Primo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Primo Esercizio
    Progettare una funzione che accetta un numero
    indefinito di dizionari e restituisca un
    dizionario che è la concatenazione di tutti i
    dizionari indicati come parametro formale
    alla funzione. Scrivete uno script che utilizzi
    tale funzione.
    Esempio:
        diz1 = {'v1':1,'v2':2,'v3':3}
        diz2 = {'v4':4,'v5':5,'v6':6}
        diz3 = {'v7':7,'v8':8}
        Dizionario restituito:
        {'v1': 1, 'v2': 2, 'v3': 3, 'v4': 4,
        'v5': 5, 'v6': 6, 'v7': 7, 'v8': 8}

"""


def concatenaDizionari1(*dizionari: (dict)) -> dict:
    """
    Funzione: concatenaDizionari1
    ** Prima implementazione **
    Concatena gli n dizionari inseriti
    come parametri formali alla funzione.
    stringa in input alla funzione.
    Parametri formali:
        lista di dizioari
    Valore di ritorno:
        dict diz -> concatenazione dei dizionario
    """

    # Inizializzo il dizionario di output
    dizionario = {}

    # Loop di concatenazione
```

```
    for diz in dizionari:
        dizionario.update(diz)

    return dizionario


def concatenaDizionari2(*dizionari: (dict)) -> dict:
    """
    Funzione: concatenaDizionari1
    ** Seconda implementazione **
    Concatena gli n dizionari inseriti
    come parametri formali alla funzione.
    stringa in input alla funzione.
    Parametri formali:
        lista di dizioari
    Valore di ritorno:
        dict diz -> concatenazione dei dizionario
    """

    # Inizializzo il dizionario di output
    dizionario = {}

    # Loop di concatenazione
    for diz in dizionari:
        for key, value in diz.items():
            dizionario[key] = value

    return dizionario


def concatenaDizionari3(*dizionari: (dict)) -> dict:
    """
    Funzione: concatenaDizionari1
    ** Terza implementazione **
    ** (operatore | inserito dalla
    ** versione 3.9
    ** https://peps.pyt.org/pep-0584/hon
    Concatena gli n dizionari inseriti
    come parametri formali alla funzione.
    stringa in input alla funzione.
    Parametri formali:
        lista di dizionari
    Valore di ritorno:
        dict diz -> concatenazione dei dizionario
    """
```

```
    # Inizializzo il dizionario di output
    dizionario = {}

    # Loop di concatenazione
    for diz in dizionari:
        dizionario |= diz

    return dizionario


"""
Programma principale
Si prova la funzione di concatenazione dei
dizionari passati alla funzione stessa.
"""

# Sezione di input (input hard-coded)
diz1 = {"v1": 1, "v2": 2, "v3": 3}
diz2 = {"v4": 4, "v5": 5, "v6": 6}
diz3 = {"v7": 7, "v8": 8}

# Elaborazione
dizOUT1 = concatenaDizionari1(diz1, diz2, diz3)
dizOUT2 = concatenaDizionari2(diz1, diz2, diz3)
dizOUT3 = concatenaDizionari3(diz1, diz2, diz3)

# Sezione di output
print("1 Implementazione", dizOUT1, sep="\n")
print("2 Implementazione", dizOUT2, sep="\n")
print("3 Implementazione", dizOUT3, sep="\n")
```

Secondo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo Esercizio
    Scrivete uno script Python per generare e
    stampare un dizionario che contenga un
    numero (compreso tra 1 e n) nella forma
    (x, x*x).

    Esempio:
        n = 4
        Dizionario: {1: 1, 2: 4, 3: 9, 4: 16}

"""

def creaDizionario1(n: int) -> dict:
    """
    Funzione: creaDizionario
    ** Prima implementazione **
    Crea dizionario le cui chiavi sono
    numeri naturali, mentre i valori
    sono i quadrati delle chiavi.
    Parametri formali:
        int n -> numero limite
    Valore di ritorno:
        dict diz -> dizionario dei quadrati
    """

    # Inizializzo il dizionario di output
    quadrati = {}

    # Loop di concatenazione
    for key in range(1, n + 1):
        quadrati[key] = key * key

    return quadrati

def creaDizionario2(n: int) -> dict:
    """
    Funzione: creaDizionario
    ** Seconda implementazione **
```

```
    ** Dictionary comprehension
    Crea dizionario le cui chiavi sono
    numeri naturali, mentre i valori
    sono i quadrati delle chiavi.
    Parametri formali:
        int n -> numero limite
    Valore di ritorno:
        dict diz -> dizionario dei quadrati
    """

    # Inizializzo il dizionario di output
    quadrati = {x:x*x for x in range(1,n+1)}

    return quadrati


"""
Programma principale
Si prova la funzione di concatenazione dei
dizionari passati alla funzione stessa.
"""

# Sezione di input (input hard-coded)
n = 5

# Elaborazione
dizOUT1 = creaDizionario1(n)
dizOUT2 = creaDizionario2(n)

# Sezione di output
print("1 Implementazione", dizOUT1, sep="\n")
print("2 Implementazione", dizOUT2, sep="\n")
```

Terzo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Terzo Esercizio
    Scrivete un programma Python per ottenere il
    valore massimo e minimo in un dizionario.

"""

# Prima soluzione SBAGLIATA!

# Inserimento dati
diz1 = {'v1':3,'v2':2,'v3':1}

# Elaborazione
min1 = min(diz1.values())
max1 = max(diz1.values())

# Output
print(min1,max1)


# Seconda soluzione CORRETTA!
# I valori da restituire devono essere chiavi
# e non i valori del dizionario!!

def keyfun(k:any):
     '''
     Funzione: keyfun
     Funzione che aiuta la ricerca del valore
     minimo e massimo contenuti in un dizionario
     Parametri formali:
          k : chiave del dizionario considerato
     Valore di ritorno:
          valore corrispondente alla chiave
          parametro di input.
     '''
     return diz1[k]

max1 = max(diz1.keys(), key=keyfun)
min1 = min(diz1.keys(), key=keyfun)
```

```
# Output
print(min1,":",diz1[min1],max1,":",diz1[max1])
```

Quarto Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quarto Esercizio
    Scrivete un programma Python per rimuovere i
    duplicati dal dizionario.

"""


def eliminaDuplicati1(diz: dict) -> dict:
    """
    Funzione: eliminaDuplicati
    ** Prima implementazione **
    Elimina i valori duplicati di un dizionario
    utilizzando una lista di supporto.
    Parametri formali:
        dict diz -> dizionario di input
    Valore di ritorno:
        dict diz -> dizionario di output
    """

    # Inizializzo il dizionario di output
    dizionario = dict()
    # Inizializzo la lista di supporto
    l = list()

    # Loop di eliminazione duplicati
    for key, value in diz.items():
        # verifico se valore non presente e
        # quindi non duplicato in dizionario
        if value not in l:
            dizionario[key] = value
            # memorizza il valore
            l.append(value)

    return dizionario


def eliminaDuplicati2(diz: dict) -> dict:
    """
```

```
    Funzione: eliminaDuplicati
    ** Seconda implementazione **
    Elimina i valori duplicati di un dizionario
    senza utilizzare una lista di supporto.
    Parametri formali:
        dict diz -> dizionario di input
    Valore di ritorno:
        dict diz -> dizionario di output
    """

    # Inizializzo il dizionario di output
    dizionario = dict()

    # Loop di eliminazione duplicati
    for key, value in diz.items():
        # verifico se valore non presente e
        # quindi non duplicato in dizionario
        if value not in dizionario.values():
            dizionario[key] = value

    return dizionario


"""
Programma principale
Si prova la funzione di eliminazione dei
valori duplicati in un dizionario.
"""

# Sezione di input (input hard-coded)
diz = {"v1": 1, "v2": 3, "v3": 3}

# Elaborazione
dizOUT1 = eliminaDuplicati1(diz)
dizOUT2 = eliminaDuplicati2(diz)

# Sezione di output
print("1 Implementazione", dizOUT1, sep="\n")
print("2 Implementazione", dizOUT2, sep="\n")
```

Quinto Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quinto Esercizio
    Scrivete un programma Python per creare un
    dizionario da una stringa. Le lettere della
    stringa rappresentano le chiavi, i valori
    rappresentano le occorrenze della chiave
    nella stringa
    Esempio
    stringa = “ciao mamma”'
    Dizionario:
    {'c': 1, 'i': 1, 'a': 3, 'o': 1, ' ':1,'m': 3}

"""


def contaFrequenza1(frase: str) -> dict:
    """
    Funzione: contaFrequenza
    ** Prima implementazione **
    Crea un dizionario le cui chiavi sono
    i caratteri della frase, i valori le
    frequenze delle chiavi all'interno della
    frase.
    Parametri formali:
        str frase -> frase di input
    Valore di ritorno:
        dict diz -> dizionario delle frequenze
    """

    # Inizializzo il dizionario delle frequenze
    frequenza = dict()

    # Loop di conteggio occorrenze caratteri
    for carattere in frase:
        if carattere in frequenza:
            frequenza[carattere] += 1
        else:
            frequenza[carattere] = 1
```

```
    return frequenza


def contaFrequenza2(frase: str) -> dict:
    """
    Funzione: contaFrequenza
    ** Seconda implementazione **
    Crea un dizionario le cui chiavi sono
    i caratteri della frase, i valori le
    frequenze delle chiavi all'interno della
    frase.
    Parametri formali:
        str frase -> frase di input
    Valore di ritorno:
        dict diz -> dizionario delle frequenze
    """

    # Inizializzo il dizionario delle frequenze
    frequenza = dict()

    # Loop di conteggio occorrenze caratteri
    for carattere in frase:
        frequenza[carattere] = frequenza.get(carattere, 0) + 1

    return frequenza


def contaFrequenza3(frase: str) -> dict:
    """
    Funzione: contaFrequenza
    ** Terza implementazione **
    Crea un dizionario le cui chiavi sono
    i caratteri della frase, i valori le
    frequenze delle chiavi all'interno della
    frase.
    Parametri formali:
        str frase -> frase di input
    Valore di ritorno:
        dict diz -> dizionario delle frequenze

    ** ATTENZIONE
    ** Per ottimizzare il codice conviene
    ** eliminare i caratteri duplicati nella
    ** frase!
```

```
    """

    # Per ottimizzare il codice
    # uso una procedura per eliminare
    # i caratteri duplicati nella frase
    # Posso utilizzare la struttura informativa set
    # caratteriUnici=set(frase)

    # Inizializzo il dizionario delle frequenze
    frequenza = dict()

    # Loop di conteggio occorrenze caratteri
    for carattere in frase:
        frequenza[carattere]=frase.count(carattere)

    return frequenza

def contaFrequenza4(frase: str) -> dict:
    """
    Funzione: contaFrequenza
    ** Quarta implementazione **
    Crea un dizionario le cui chiavi sono
    i caratteri della frase, i valori le
    frequenze delle chiavi all'interno della
    frase.
    Parametri formali:
        str frase -> frase di input
    Valore di ritorno:
        dict diz -> dizionario delle frequenze

    ** ATTENZIONE
    ** Per ottimizzare il codice conviene
    ** eliminare i caratteri duplicati nella
    ** frase!

    """

    # Per ottimizzare il codice
    # uso una procedura per eliminare
    # i caratteri duplicati nella frase
    # Posso utilizzare la struttura informativa set
    # caratteriUnici=set(frase)

    # Dictionary comprehension
    frequenza = {carattere: frase.count(carattere) for carattere in
frase}
```

```
    return frequenza


"""
Programma principale
Si prova la funzione di conteggio frequenze
caratteri in una frase.
"""

# Sezione di input (input hard-coded)
frase = "ciao mamma"

# Elaborazione
frequenza1 = contaFrequenza1(frase)
frequenza2 = contaFrequenza2(frase)
frequenza3 = contaFrequenza3(frase)
frequenza4 = contaFrequenza4(frase)

# Sezione di output
print("1 Implementazione", frequenza1, sep="\n")
print("2 Implementazione", frequenza2, sep="\n")
print("3 Implementazione", frequenza3, sep="\n")
print("4 Implementazione", frequenza4, sep="\n")
```